



AULA 03 • BACKEND

Orientação a Objetos em Java

+ Tópicos Especiais: Object, Construtores, Collections e Exceções

Teoria em bloco único → Prática guiada em cima do projeto exemplo-oop

Como a aula vai fluir

1

Teoria — OOP em Java

Classes, objetos, herança, encapsulamento, interfaces e polimorfismo — revisão rápida e direto ao ponto.

2

Teoria — Tópicos Especiais

Classe Object, construtores, Collections e tratamento de exceções.

3

Prática — Mão na massa

Vamos abrir o projeto exemplo-oop e evoluir o código juntos, exercício por exercício.

PARTE 1

Orientação a Objetos em Java

Revisão do conceito de OOP aplicada com o projeto exemplo-oop

Por que existe a Orientação a Objetos?

- Sem OOP, um programa vira uma pilha de variáveis soltas e funções separadas — difícil de saber o que pertence a quê.
- Exemplo: em vez de nome, raça, dono e emitirSom() espalhados por variáveis avulsas, a OOP agrupa tudo isso dentro de um único "pacote" chamado Animal.
- Isso facilita reaproveitar código (herança), proteger dados (encapsulamento) e organizar sistemas grandes em pedaços menores e compreensíveis.
- Toda a disciplina de Backend usa OOP porque é assim que Java (e a maioria das linguagens usadas em produção) organiza o código.

✗ Sem OOP

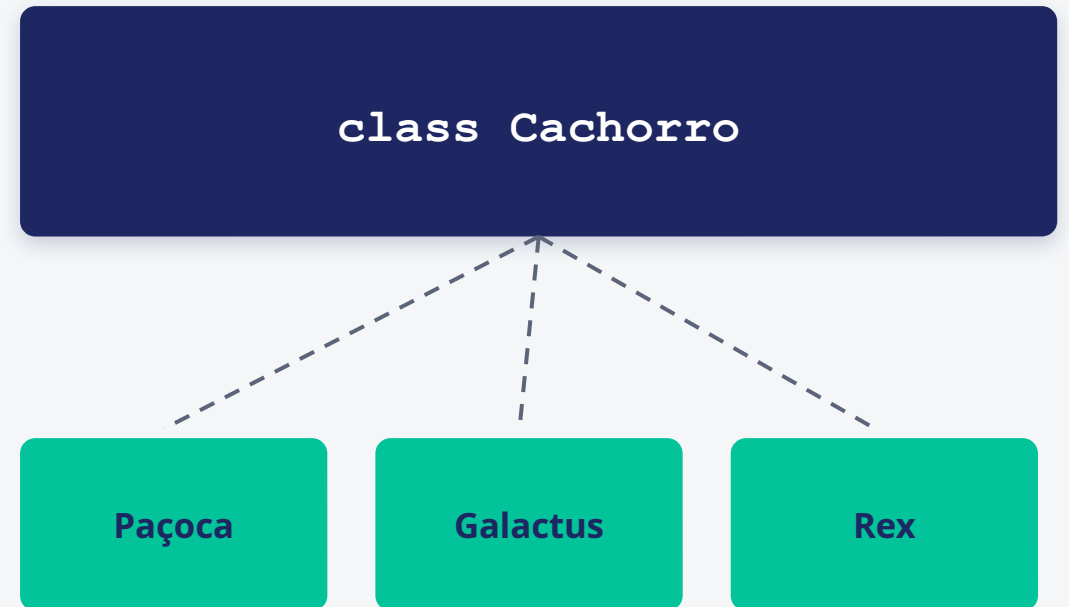
```
String nomeCachorro;  
String racaCachorro;  
String nomeGato;  
String racaGato;  
// ... tudo solto e sem relação clara
```

✓ Com OOP

```
class Animal {  
    String nome;  
    String raca;  
    // atributos e métodos juntos,  
    // organizados num único lugar  
}
```

O que é Orientação a Objetos?

- OOP (Object Oriented Programming) organiza o código em torno de objetos, não de funções soltas.
- Cada objeto representa algo do mundo real (ou de um conceito abstrato) com dados (atributos) e comportamentos (métodos).
- A classe é o "molde" — o objeto é a coisa concreta criada a partir dele. Chamamos cada objeto criado de instância da classe.
- Exemplo clássico: a classe Cachorro descreve cor, raça e peso (atributos) e correr, comer, latir (métodos).
- Cada instância guarda seus próprios valores: o Cachorro "Paçoca" e o Cachorro "Galactus" compartilham a mesma estrutura, mas têm dados diferentes.



1 classe → vários objetos (instâncias), cada um com seus próprios valores.

Classes, atributos, métodos e this

- Atributos = variáveis dentro da classe. Métodos = funções dentro da classe.
- `new NomeDaClasse()` cria um novo objeto (instância) daquela classe.
- `this` refere-se ao próprio objeto dentro de um método — útil quando o parâmetro tem o mesmo nome do atributo.

```
Animal.java

public class Animal {
    private String nome;

    public void setNome(String nome) {
        // this.nome = atributo da classe
        // nome      = parâmetro recebido
        this.nome = nome;
    }
}
```

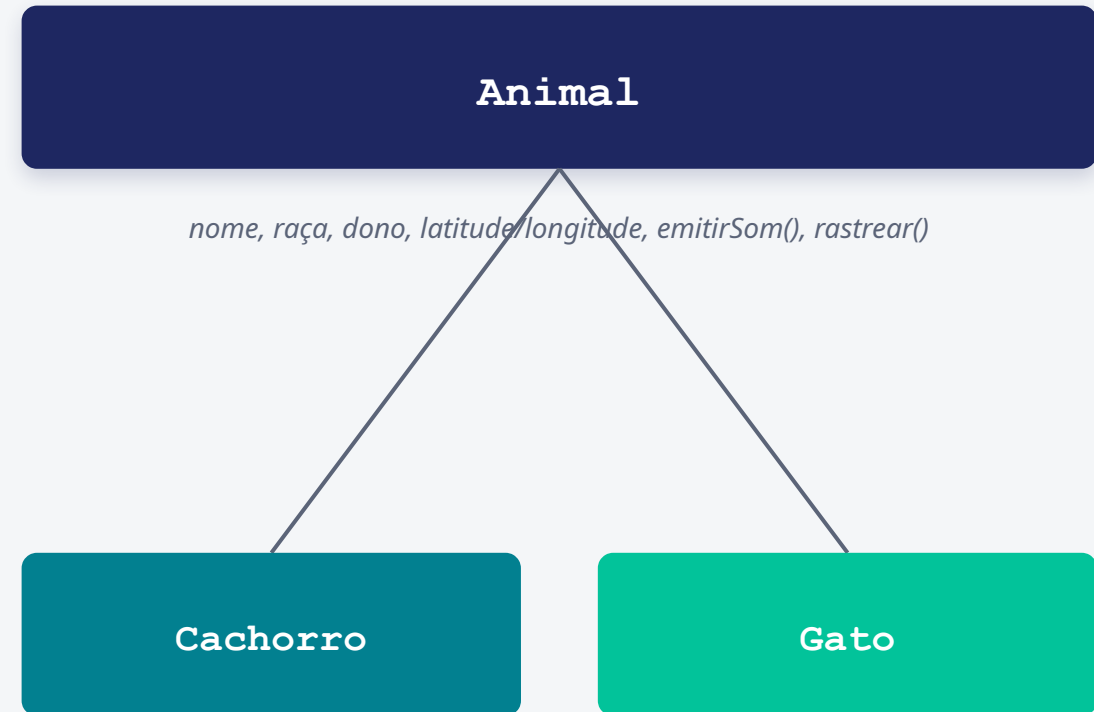
Herança — extends

- Uma classe pode herdar atributos e métodos de outra usando extends.
- No nosso projeto: Cachorro e Gato herdam tudo de Animal (nome, raça, dono, posição, emitirSom...).
- `super(...)` chama o construtor da superclasse — é assim que a subclasse aproveita a inicialização que o pai já sabe fazer, sem reescrevê-la.
- Fora de um construtor, `super.metodo()` chama a versão do método implementada no pai, mesmo que você tenha sobrescrito esse método na subclasse.

```
Animal.java / Cachorro.java

public class Animal {
    private String nome;
    public Animal(String nome) { this.nome = nome; }
}

public class Cachorro extends Animal {
    public Cachorro(String nome) { super(nome); }
}
```



Encapsulamento e modificadores de acesso

- Encapsulamento = esconder os atributos com `private` e liberar acesso controlado via getters e setters.
- `private`: só a própria classe acessa.
- `protected`: acessa dentro do mesmo package (package é apenas uma "pasta" que agrupa classes relacionadas dentro do projeto).
- `public`: acessa de qualquer lugar.
- Por que fazer isso? Porque assim a classe controla suas próprias regras — ninguém de fora consegue colocar um nome vazio ou um valor inválido diretamente.

```
Animal.java

public class Animal {
    private String nome;

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        if (nome.isEmpty()) return;
        this.nome = nome;
    }
}
```


Getters e Setters

- Getter = método que só devolve o valor de um atributo privado. Setter = método que só altera esse valor (podendo validar antes).
- A convenção é nomear como `getNomeDoAtributo()` e `setNomeDoAtributo(valor)` — começando com "get" e "set".
- Exceção da convenção: atributos boolean geralmente usam `isNomeDoAtributo()` no lugar de `getNomeDoAtributo()` (ex.: `isAtivo()` em vez de `getAtivo()`).
- Isso não é regra do compilador — é convenção da comunidade Java, mas frameworks e outros programadores esperam esse padrão.

```
Funcionario.java

public class Funcionario {
    private String nome;
    private double salario;
    private boolean ativo;

    public String getNome() { return nome; }
    public void setNome(String nome) {
        this.nome = nome;
    }

    public void setSalario(double salario) {
        if (salario < 0) return;
        this.salario = salario;
    }

    // boolean usa "is", não "get"
    public boolean isAtivo() { return ativo; }
}
```

Interfaces e Polimorfismo

- Interface define métodos que quem a implementa é obrigado a escrever (implements). É como um contrato: "toda classe que assinar este contrato precisa ter estes métodos".
- Rastreavel é uma interface com o método rastrear().
- Animal (e suas subclasses) e Robo implementam Rastreavel — mesmo sendo bem diferentes entre si.
- Polimorfismo: uma lista de Rastreavel pode guardar Cachorro, Gato e Robo ao mesmo tempo, e cada um responde rastrear() do seu jeito.

 *Analogia: um controle remoto de TV e um de ar-condicionado são bem diferentes por dentro, mas os dois têm um botão "ligar". Polimorfismo é isso: comportamentos diferentes por trás do mesmo "botão" (método).*

Rastreavel.java / Robo.java

```
public interface Rastreavel {  
    public String rastrear();  
}  
  
public class Robo implements Rastreavel {  
    private String posicao;  
    public String rastrear() {  
        return posicao;  
    }  
}
```

Classes x Classes Abstratas x Interfaces

Estrutura	new	Métodos	Atributos	Herança	Palavra-chave
Classes regulares	Sim	Sim	Sim	Sim	class
Classes abstratas	Não	Sim	Sim	Sim	abstract class
Interfaces	Não	Sim	Não	Não	interface

💡 *Interface = 100% abstrata (nenhum método pronto). Classe abstrata pode misturar métodos prontos e métodos abstratos.*

PARTE 2

Tópicos Especiais em Java

Object, construtores, Collections e tratamento de exceções

A classe Object

- Toda classe em Java herda — de forma implícita, mesmo sem você escrever `extends` — da classe `Object`.
- `toString()`: sem sobrescrever, o padrão é algo pouco legível como `Animal@1a2b3c`. Sobrescrevendo (com `@Override`), você decide o que aparece no console.
- `equals()`: por padrão, compara se é o mesmo objeto na memória. Sobrescrevendo, você decide o que conta como "igual" (ex.: mesmo nome).
- Regra de ouro: se você sobrescrever `equals()`, sobrescreva `hashCode()` também — `HashMap` e `HashSet` usam os dois por trás dos panos para funcionar direito.

```
Animal.java

public class Animal {
    private String nome;
    private String raca;

    @Override
    public String toString() {
        return nome + " (" + raca + ")";
    }

    @Override
    public boolean equals(Object obj) {
        if (!(obj instanceof Animal)) return false;
        Animal outro = (Animal) obj;
        return this.nome.equals(outro.nome);
    }
}

System.out.println(galactus);
// Galactus (Pincher) ✓ não mais Animal@1a2b3c
```

Construtores

- Construtor = método especial chamado no momento do new, com o mesmo nome da classe e sem tipo de retorno.
- Se você não escrever nenhum, o Java gera um construtor padrão vazio (NoArgsConstructor). Ao escrever qualquer construtor, esse padrão automático deixa de existir.
- Assim como métodos, construtores podem ser sobrecarregados: a mesma classe pode ter várias versões de construtor, cada uma com parâmetros diferentes.
- this(...) dentro de um construtor chama outro construtor da mesma classe — útil para não repetir lógica de inicialização.

```
Pessoa.java

public class Pessoa {
    private String nome;
    private int idade;

    // construtor sem argumentos
    public Pessoa() {
        this("Sem nome", 0);
    }

    // construtor com todos os argumentos
    public Pessoa(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }
}

Pessoa p1 = new Pessoa();
Pessoa p2 = new Pessoa("Ana", 30);
```

Java não tem "destrutor" como C++

A sintaxe `~NomeClasse()` é de C++, não existe em Java. Se aparecer em algum material antigo, é um erro de cópia — vale ajustar.

- Em Java, a memória é gerenciada pelo Garbage Collector (GC) automaticamente — no próximo slide explicamos exatamente o que ele faz.
- Existe o método `finalize()` (herdado de `Object`), mas é deprecated — não deve ser usado.
- Na prática: você não escreve código para "destruir" um objeto em Java. Só deixa de referenciá-lo.

```
conceito.md

// ✗ Isso NÃO existe em Java:
// ~Pessoa() { ... }

// ✓ O Garbage Collector cuida disso
// sozinho, em segundo plano.
```

O que é o Garbage Collector?

- Toda vez que você faz `new Animal()`, o Java reserva um espacinho de memória (RAM) pra guardar aquele objeto.
- Uma variável (como o `galactus`) não é o objeto — ela é uma referência, um "post-it" que aponta pra onde o objeto está guardado na memória.
- Quando nenhuma variável aponta mais para um objeto (ex.: a variável saiu de escopo, ou recebeu outro valor), aquele objeto vira "lixo" — ninguém mais consegue acessá-lo.
- O Garbage Collector ("coletor de lixo") é uma parte do Java que roda em segundo plano, encontra esses objetos sem dono e libera a memória automaticamente.
- É por isso que você nunca precisa (e nem consegue) escrever código para "destruir" um objeto — diferente de linguagens como C++.

```
Animal galactus = new Animal();
```

objeto `Animal` na memória

```
galactus = null; // ninguém mais aponta pra cá
```



Garbage Collector recolhe a memória

O que significam os <> (Generics)

- Você vai ver bastante coisa como `ArrayList<Rastreavel>` ou `HashMap<String, Rastreavel>` — os <> não são decoração, eles dizem qual tipo de dado vai dentro daquela estrutura.
- É como rotular uma caixa: "essa caixa só guarda `Animal`" evita que alguém guarde uma `String` ali dentro por engano.
- Sem generics, o Java deixaria colocar qualquer coisa lá dentro e o erro só apareceria em tempo de execução — bem mais difícil de achar.
- Você vai ler <T> como "tipo genérico" em algumas documentações — mas no nosso dia a dia, geralmente é um tipo concreto mesmo, como <String> ou <Animal>.

```
Main.java

// Lista que só aceita Strings
List<String> nomes = new ArrayList<>();
nomes.add("Galactus");
// nomes.add(123); ❌ não compila!

// Map: chave String, valor Animal
Map<String, Animal> animais = new HashMap<>();
animais.put("Galactus", galactus);

Animal a = animais.get("Galactus");
// já vem como Animal, sem precisar
// converter (cast) manualmente
```

Collections

😞 Por que não usar só um array? Porque array tem tamanho fixo — depois de criado, não dá pra adicionar ou remover posições. Collections resolvem isso.



List

Ordenada, aceita duplicados

`ArrayList`, `LinkedList`



Set

Sem elementos duplicados

`HashSet`



Queue

Fila — organiza por ordem de processamento

`PriorityQueue`



Map

Chave → valor (não é Collection, mas é tratado junto)

`HashMap`

Usando uma List de verdade

- ArrayList é a implementação de List mais usada no dia a dia — funciona como uma "gaveta" que cresce sob demanda.
- add() insere, get(indice) busca pela posição, contains() verifica se existe, remove() tira um elemento, size() diz quantos existem.
- O for-each (for (Tipo item : lista)) percorre todos os elementos sem precisar controlar índice manualmente.
- HashSet e HashMap seguem a mesma lógica de uso, mudando apenas o que cada um garante (sem duplicados / chave-valor).

```
Main.java

List<String> nomes = new ArrayList<>();
nomes.add("Galactus");
nomes.add("Fumaça");
nomes.add("R2D2");

System.out.println(nomes.contains("Fumaça"));
// true

nomes.remove("R2D2");
System.out.println(nomes.size());
// 2

for (String nome : nomes) {
    System.out.println(nome);
}
// Galactus
// Fumaça
```

O que é uma exceção?

- Exceção é um "aviso de erro" que o Java lança quando algo impede o código de continuar normalmente (ex.: dividir por zero, acessar posição que não existe).
- Se ninguém tratar a exceção, o programa para imediatamente e imprime um stack trace (o "rastro" de onde o erro aconteceu).
- Checked: verificadas em tempo de compilação — o próprio compilador te obriga a tratar ou declarar com throws (ex.: IOException, ao mexer com arquivos).
- Unchecked: só aparecem em tempo de execução, geralmente por bugs de lógica (ex.: ArrayIndexOutOfBoundsException).

```
Main.java

int[] numeros = {1, 2, 3};
System.out.println(numeros[5]);

// ✨ Programa para e imprime:
// Exception in thread "main"
// java.lang.ArrayIndexOutOfBoundsException
//   Exception: Index 5 out of
//   bounds for length 3
//   at Main.main(Main.java:2)

// Sem try/catch, tudo o que vier
// depois dessa linha NÃO executa.
```

Tratando exceções: throw e try-catch

- throw dispara uma exceção manualmente quando uma regra do seu próprio código é quebrada.
- try/catch captura o erro e evita que o programa pare de rodar — o que está dentro do catch só roda se der erro no try.
- finally roda sempre — deu erro ou não. É o lugar certo para liberar recursos (fechar um arquivo, uma conexão, etc.).
- getMessage() devolve o texto que você passou ao criar a exceção.

```
Calculadora.java

public int dividir(int a, int b) {
    if (b == 0) {
        throw new ArithmeticException(
            "Impossível dividir por zero!");
    }
    return a / b;
}

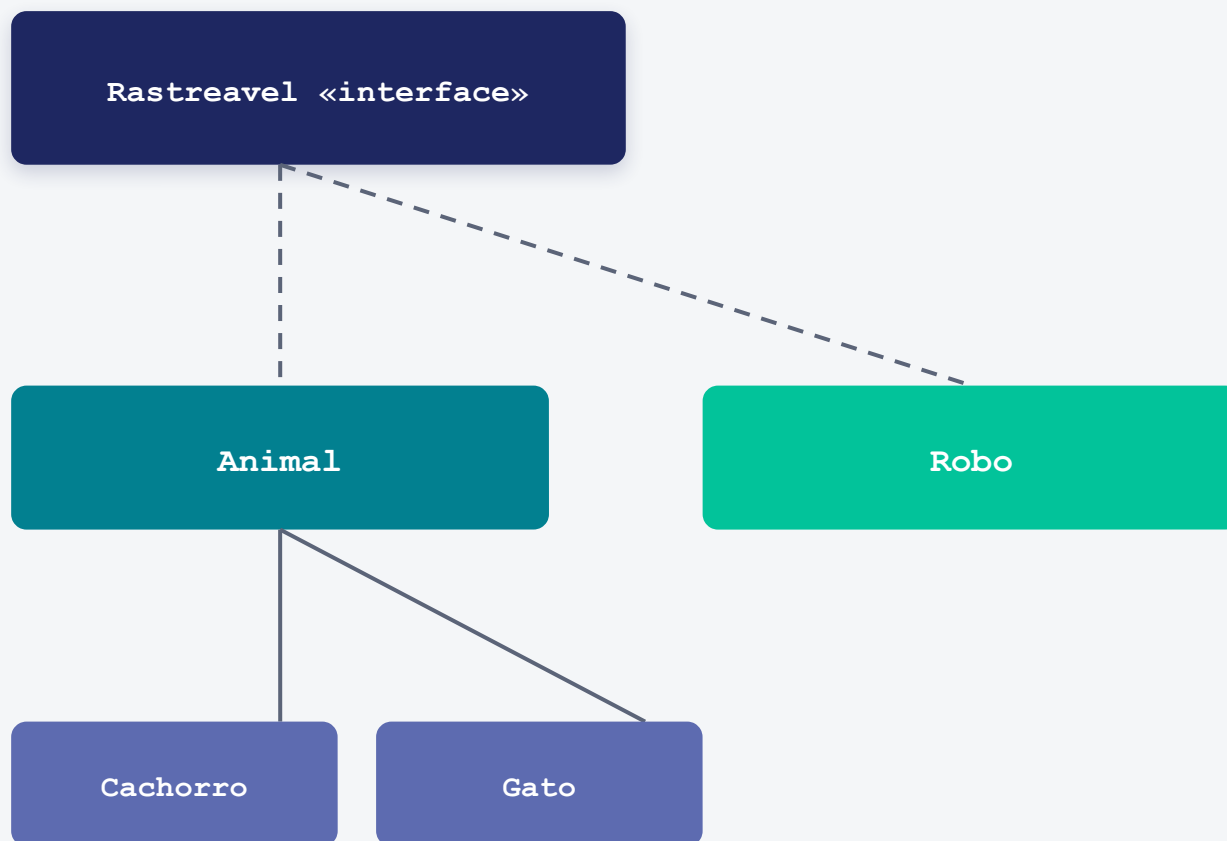
try {
    dividir(10, 0);
} catch (ArithmeticException e) {
    System.out.println(e.getMessage());
} finally {
    System.out.println("Fim da tentativa.");
}
```

PARTE 3

Prática — Mão na Massa

Vamos evoluir o projeto exemplo-oop juntos, passo a passo

exemplo-oop: o que já existe



- Animal implementa Rastreavel e tem nome, raça, dono, latitude/longitude (com getters/setters).
- Cachorro e Gato herdam de Animal (extends) e não adicionam nada ainda.
- Robo implementa Rastreavel de forma independente (nome, posição).
- Main.java já monta uma `ArrayList<Rastreavel>` com tipos diferentes — polimorfismo em ação.

EX. 1

ABSTRAÇÃO + POLIMORFISMO OBRIGATÓRIO

Animal vira classe abstrata

🎯 *Objetivo: Impedir que Animal seja instanciado direto e obrigar cada subclasse a definir seu próprio som.*

- Adicione `abstract` em "class Animal" e remova o corpo de `emitirSom()`, deixando-o `abstract`.
- Em `Cachorro`, implemente `emitirSom()` imprimindo "Au au!". Em `Gato`, "Miau!".
- Tente descomentar `new Animal()` no Main — o compilador deve recusar.
- Rode o Main e confirme: cada subtipo emite seu próprio som ao percorrer a lista.

```
Animal.java / Cachorro.java

public abstract class Animal implements Rastreavel {
    private String nome;
    // ...outros atributos

    public abstract void emitirSom();
}

public class Cachorro extends Animal {
    @Override
    public void emitirSom() {
        System.out.println("Au au!");
    }
}

// new Animal(); ❌ não compila!
```


EX. 2

CLASSE DE EXCEÇÃO PRÓPRIA + THROWS

Exceção personalizada

 *Objetivo: Criar sua própria exceção e validar coordenadas reais antes de salvar a posição.*

- Crie `PosicaoInvalidaException` extends `Exception` (construtor recebe a mensagem).
- Em `setPosicao(int lat, int lon)`, valide: latitude entre -90 e 90, longitude entre -180 e 180.
- Se inválido, lance (throw) a sua exceção com uma mensagem clara.
- No Main, chame `setPosicao` dentro de um try/catch e trate o erro sem derrubar o programa.

```
PosicaoInvalidaException.java

public class PosicaoInvalidaException extends Exception {
    public PosicaoInvalidaException(String msg) {
        super(msg);
    }
}

public void setPosicao(int lat, int lon)
    throws PosicaoInvalidaException {
    if (lat < -90 || lat > 90 ||
        lon < -180 || lon > 180) {
        throw new PosicaoInvalidaException(
            "Coordenadas inválidas!");
    }
    this.latitude = lat;
    this.longitude = lon;
}
```

EX. 3

COLLECTIONS: HASHMAP + BUSCA POR CHAVE

De List para Map

 *Objetivo: Trocar a estrutura de dados para permitir localizar um rastreável instantaneamente pelo nome.*

- No Main, troque `ArrayList<Rastreavel>` por `HashMap<String, Rastreavel>`, usando o nome como chave.
- Crie um método `buscarPorNome(String nome)` que devolve o `Rastreavel` correspondente (ou null).
- Busque um animal específico pelo nome e imprima o resultado de `rastrear()`.
- Discussão: por que buscar num Map é mais rápido do que percorrer uma List inteira?

```
Main.java

HashMap<String, Rastreavel> rastreaveis = new HashMap<>();
rastreaveis.put(galactus.getNome(), galactus);
rastreaveis.put(robo.getNome(), robo);

Rastreavel encontrado = rastreaveis.get("Galactus");

if (encontrado != null) {
    System.out.println(encontrado.rastrear());
} else {
    System.out.println("Não encontrado!");
}
```

EX. 4

INTERFACE COMPARABLE + COLLECTIONS.SORT

Ordenando com Comparable

 *Objetivo: Deixar os animais em ordem alfabética de nome antes de exibí-los.*

- Faça Animal implementar Comparable<Animal>.
- Implemente compareTo(Animal outro) usando this.getNome().compareTo(outro.getNome()).
- Monte uma List<Animal> com vários animais fora de ordem.
- Chame Collections.sort(lista) e imprima — deve sair em ordem alfabética.

```
Animal.java / Main.java

public class Animal implements Rastreavel,
    Comparable<Animal> {

    @Override
    public int compareTo(Animal outro) {
        return this.getNome()
            .compareTo(outro.getNome());
    }
}

List<Animal> lista = new ArrayList<>();
// ...adiciona os animais
Collections.sort(lista);
```

Desafio final: CentralDeRastreamento

 *Objetivo: Construir uma classe de serviço que junta tudo: cadastro sem duplicidade, exceção própria e relatório ordenado.*

- Crie CentralDeRastreamento com um `HashMap<String, Rastreavel>` interno (privado).
- Método `cadastrar(String nome, Rastreavel r)`: lança `NomeDuplicadoException` se o nome já existir.
- Método `relatorio()`: ordena os nomes (`Collections.sort` numa `List<String>`) e imprime nome + rastrear() de cada um.
- No Main, cadastre Cachorro, Gato, Robo e Passaro tratando a exceção com try/catch, depois chame `relatorio()`.

```
CentralDeRastreamento.java

public class CentralDeRastreamento {
    private HashMap<String, Rastreavel> dados
        = new HashMap<>();

    public void cadastrar(String nome, Rastreavel r)
        throws NomeDuplicadoException {
        if (dados.containsKey(nome)) {
            throw new NomeDuplicadoException(nome);
        }
        dados.put(nome, r);
    }

    public void relatorio() {
        List<String> nomes = new ArrayList<>(dados.keySet());
        Collections.sort(nomes);
        for (String n : nomes)
            System.out.println(n + ": " + dados.get(n).rastrear());
    }
}
```

Glossário rápido — Orientação a Objetos (1/2)

Classe: molde que descreve os atributos e métodos de um tipo de objeto.

Objeto / Instância: a "coisa" concreta criada a partir de uma classe (via new), com valores próprios.

Atributo: uma variável que vive dentro de uma classe — um dado do objeto.

Método: uma função que vive dentro de uma classe — um comportamento do objeto.

this: dentro de um método, refere-se ao próprio objeto que está sendo usado.

Herança (extends): uma classe reaproveita atributos e métodos de outra, especializando-a.

Encapsulamento: esconder atributos (private) e liberar acesso controlado via getters/setters.

Interface (implements): um "contrato" de métodos que a classe é obrigada a implementar.

Classe abstrata: não pode ser instanciada (sem new) — serve de base para outras classes.

Polimorfismo: o mesmo método se comporta de forma diferente dependendo do objeto que o chama.

Glossário rápido — Tópicos Especiais (2/2)

Object: classe da qual toda classe Java herda implicitamente (dá toString, equals...).

Construtor: método especial chamado no momento do new, usado para inicializar o objeto.

Garbage Collector (GC): mecanismo do Java que libera automaticamente a memória de objetos sem referência.

Generics (<>): indicam qual tipo de dado uma estrutura (List, Map...) vai guardar.

Collection: estrutura para guardar vários objetos: List, Set, Queue (e Map, tratado junto).

Exceção: aviso de erro lançado quando algo impede a execução normal do programa.

Checked / Unchecked: exceções verificadas em tempo de compilação vs. em tempo de execução.

try / catch: bloco que captura e trata uma exceção sem derrubar o programa.

RECAPITULANDO

O que levamos para casa

- OOP em Java: classes, herança, encapsulamento, interfaces e polimorfismo — tudo já vivo no projeto exemplo-oop.
- Object, construtores, Collections e exceções são ferramentas do dia a dia, não só teoria.
- Próxima aula: seguimos com Avaliação N1 sobre este conteúdo.

Dever de casa

Terminar os 5 exercícios (incluindo o desafio final da Central de Rastreamento) e subir o código no repositório da disciplina até a próxima aula.